

Análisis de Requerimientos y Tecnologías Utilizadas

Introducción

Para el desarrollo del sistema **Control de Inventario S.A.S.** se analizaron los requerimientos funcionales y no funcionales con el fin de seleccionar las tecnologías más adecuadas para satisfacer las necesidades del proyecto.

La arquitectura implementada permite administrar productos, controlar entradas y salidas de inventario, almacenar la información de forma persistente mediante una base de datos NoSQL y generar reportes automáticos en formato Excel. La combinación de tecnologías utilizadas permitió desarrollar una aplicación web sencilla, funcional y escalable.

Tecnologías utilizadas

HTML5

HTML5 fue utilizado para construir la estructura de la interfaz gráfica del sistema.

Su utilización permitió:

- Crear el formulario de inicio de sesión.
- Construir el panel principal del sistema.
- Mostrar la tabla del inventario.
- Organizar los formularios de registro de productos.
- Crear los controles para registrar entradas y salidas.
- Incorporar los botones para las diferentes funcionalidades del sistema.

Gracias a HTML5 se definió la estructura sobre la cual interactúa el usuario.

CSS3

CSS3 fue empleado para diseñar la apariencia visual del sistema.

Permitió:

- Organizar los elementos de la interfaz.
- Dar formato a las tablas.
- Diseñar botones y formularios.
- Aplicar colores institucionales.
- Mejorar la experiencia del usuario.
- Crear una interfaz limpia, organizada y fácil de utilizar.

JavaScript

JavaScript constituye la lógica principal del sistema tanto en el frontend como en parte de las validaciones del proyecto.

Con JavaScript se implementó:

- Inicio de sesión.
- Registro de productos.
- Edición de productos.
- Eliminación de productos.
- Registro de entradas.
- Registro de salidas.
- Validaciones de formularios.
- Generación de reportes.
- Comunicación con el servidor mediante Fetch API.
- Actualización dinámica de la información mostrada en pantalla.

Además, permitió validar la información antes de enviarla al servidor, reduciendo errores y mejorando la experiencia del usuario.

Node.js

Node.js fue utilizado para ejecutar JavaScript del lado del servidor.

Gracias a Node.js fue posible:

- Crear la API REST.
- Recibir solicitudes del frontend.
- Procesar la lógica del negocio.
- Conectarse con MongoDB.
- Gestionar el inventario.
- Generar el reporte Excel.

Express.js

Express.js facilitó la creación del servidor web y la implementación de la API REST.

Permitió definir rutas para las principales funcionalidades del sistema, como:

- Login.
- Productos.
- Movimientos.
- Reporte Excel.

Su utilización simplificó el desarrollo del backend y permitió organizar mejor la lógica de la aplicación.

MongoDB

MongoDB fue seleccionado como sistema de gestión de bases de datos debido a su facilidad de integración con aplicaciones desarrolladas en JavaScript mediante Node.js y Mongoose.

Se trata de una base de datos NoSQL orientada a documentos, lo que permite almacenar la información en formato JSON (BSON), facilitando el manejo de los datos del inventario.

En el proyecto se implementaron dos colecciones principales:

Productos

Almacena la información correspondiente a cada producto, incluyendo:

- Nombre.
- Categoría.

- Cantidad disponible.
- Precio.

Movimientos

Almacena el historial de todas las entradas y salidas registradas en el inventario.

Cada vez que el usuario realiza una operación, el servidor consulta la base de datos, actualiza la información correspondiente y almacena los cambios de forma permanente.

El uso de MongoDB permitió mejorar la seguridad, escalabilidad y organización de la información respecto al almacenamiento mediante archivos JSON.

Mongoose

Mongoose fue utilizado como ODM (Object Document Mapper) para facilitar la comunicación entre Node.js y MongoDB.

Su utilización permitió:

- Definir modelos para las colecciones.
- Validar automáticamente los datos.
- Crear documentos.
- Actualizar registros.
- Eliminar información.
- Consultar productos y movimientos mediante métodos sencillos.

Gracias a Mongoose se redujo la complejidad de trabajar directamente con consultas de MongoDB.

ExceUS

ExceUS fue utilizado para generar automáticamente el reporte del inventario.

Permitió:

- Crear libros de Excel.
- Crear hojas de cálculo.
- Agregar encabezados.

- Insertar filas dinámicamente.
- Mostrar el historial de movimientos.
- Mostrar el estado actual del inventario.
- Descargar el reporte directamente desde el navegador.

Persistencia de datos

La persistencia de la información fue implementada mediante **MongoDB**, una base de datos NoSQL orientada a documentos.

Para organizar la información se definieron dos colecciones principales:

Productos

Almacena toda la información relacionada con el inventario, incluyendo:

- Nombre.
- Categoría.
- Cantidad disponible.
- Precio.

Cada producto es almacenado como un documento independiente dentro de la colección.

Movimientos

Registra el historial completo de entradas y salidas realizadas sobre el inventario.

Cada movimiento almacena información como:

- Fecha.
- Tipo de movimiento.
- Producto.
- Cantidad.

Cuando el usuario realiza una operación desde la aplicación, el servidor procesa la solicitud mediante Express.js, utiliza Mongoose para comunicarse con MongoDB y actualiza automáticamente la información correspondiente.

Gracias a esta arquitectura, los datos permanecen almacenados de forma permanente incluso cuando el servidor se reinicia, garantizando la integridad de la información y permitiendo futuras ampliaciones del sistema.

Arquitectura del sistema

El sistema sigue una arquitectura **Cliente–Servidor**.

El usuario interactúa con la interfaz desarrollada en HTML5, CSS3 y JavaScript.

Las solicitudes son enviadas mediante **Fetch API** hacia el servidor desarrollado con **Node.js** y **Express.js**.

El servidor procesa cada solicitud, realiza las operaciones correspondientes utilizando **Mongoose** y almacena la información en **MongoDB**.

Posteriormente, el servidor devuelve una respuesta en formato JSON al cliente, permitiendo que la interfaz actualice automáticamente la información mostrada al usuario sin necesidad de recargar la página.

Esta arquitectura facilita la separación entre la interfaz, la lógica del negocio y la persistencia de datos, mejorando el mantenimiento, la escalabilidad y la organización del proyecto.

Diagrama UML

Diagrama de clases

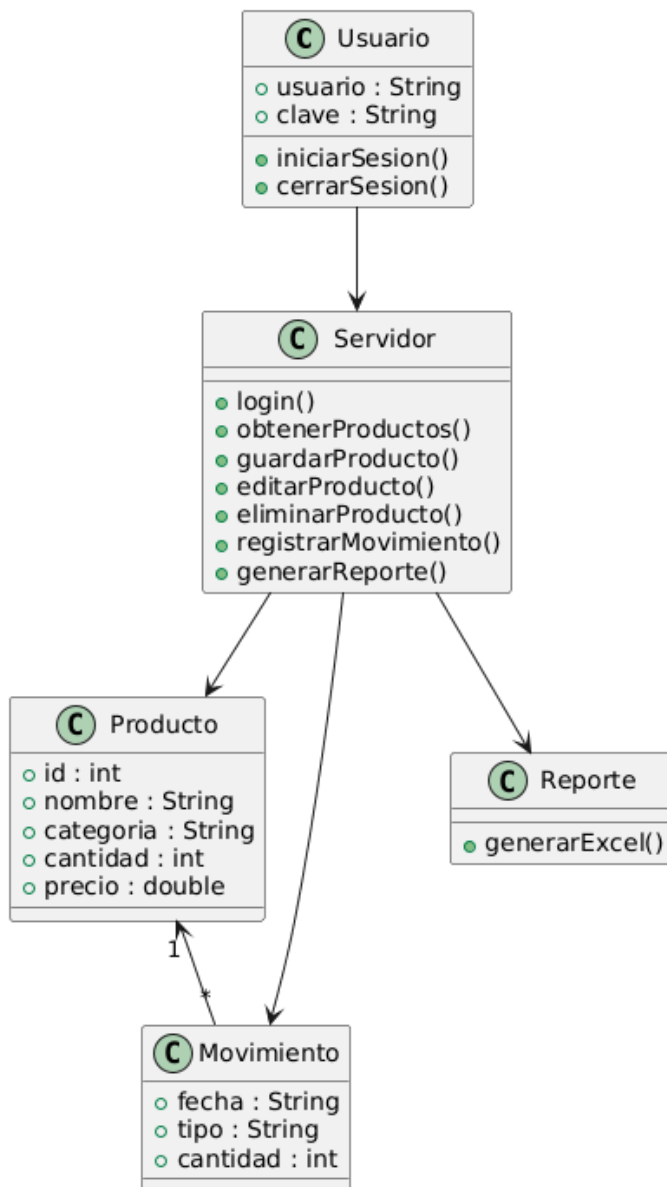
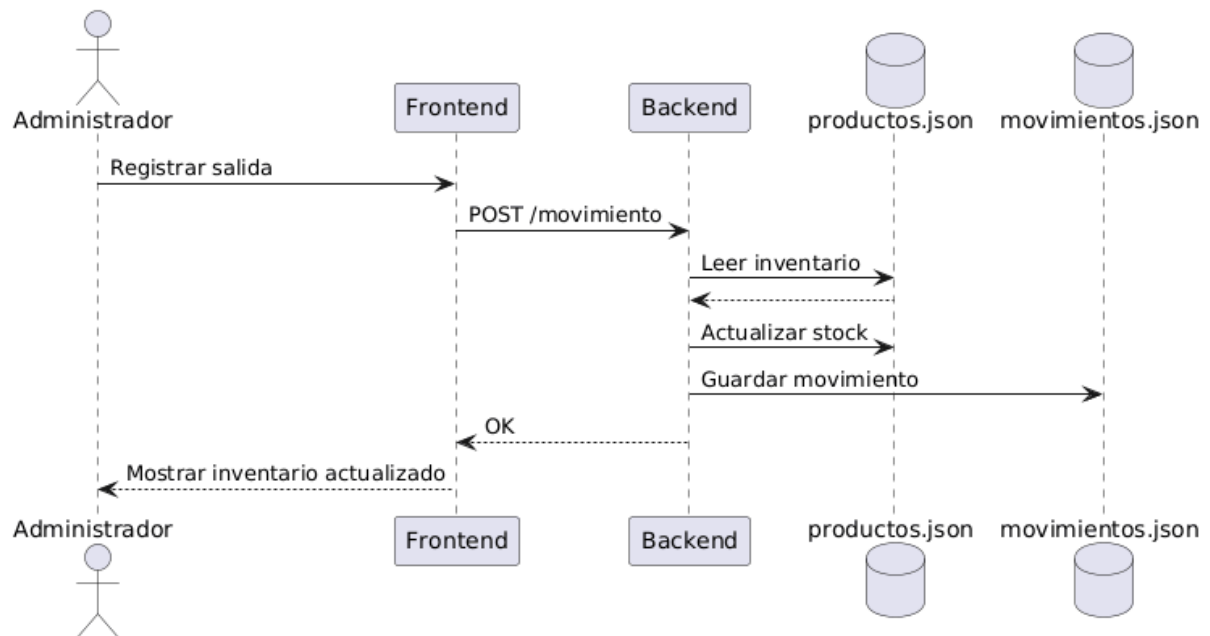
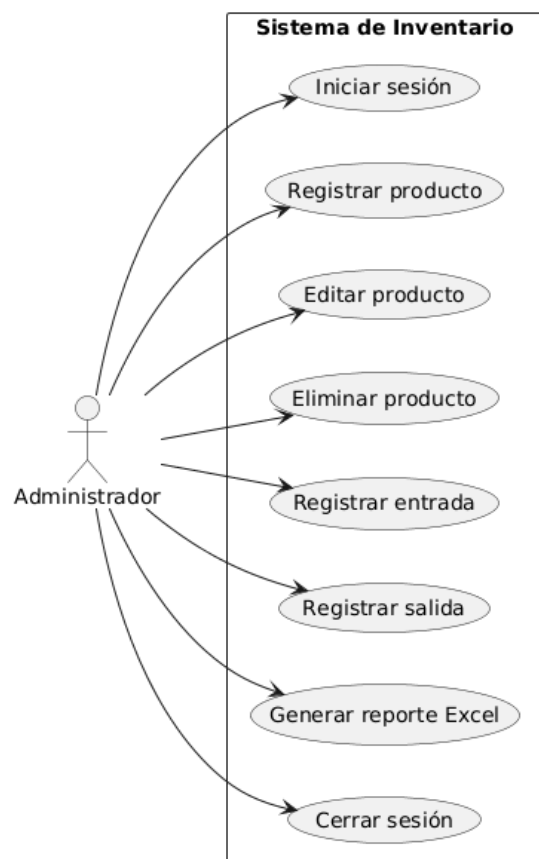


Diagrama de secuencias



Casos de uso



Conclusión

La combinación de HTML, CSS, JavaScript, Node.js, Express y ExcelJS permitió desarrollar un sistema cliente-servidor funcional para la gestión de inventarios. La arquitectura implementada facilita el mantenimiento del código, la separación de responsabilidades entre frontend y backend y la posibilidad de evolucionar el proyecto en el futuro, por ejemplo, sustituyendo la persistencia basada en archivos JSON por una base de datos relacional como MySQL sin modificar significativamente la lógica de la aplicación.